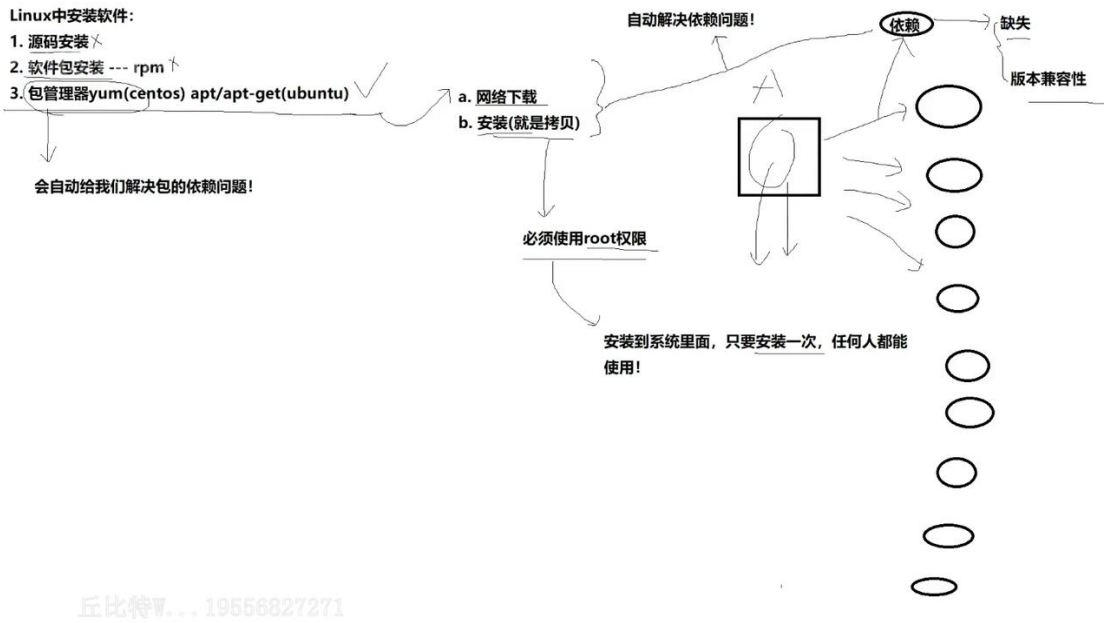
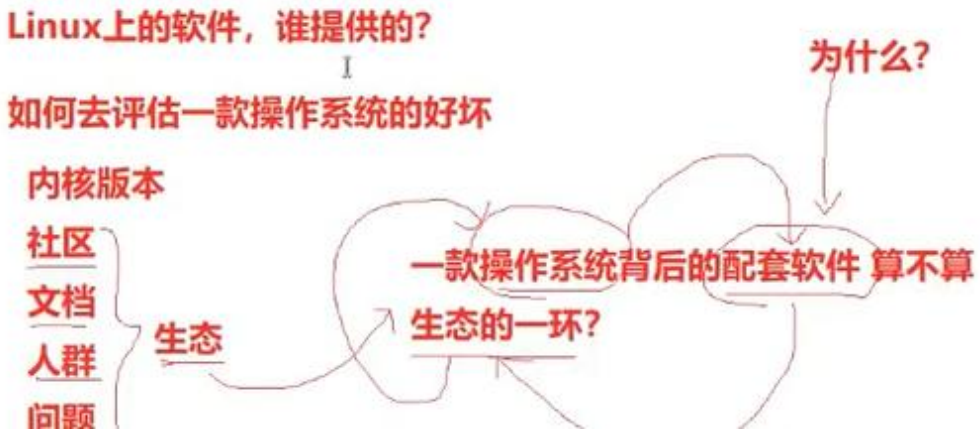
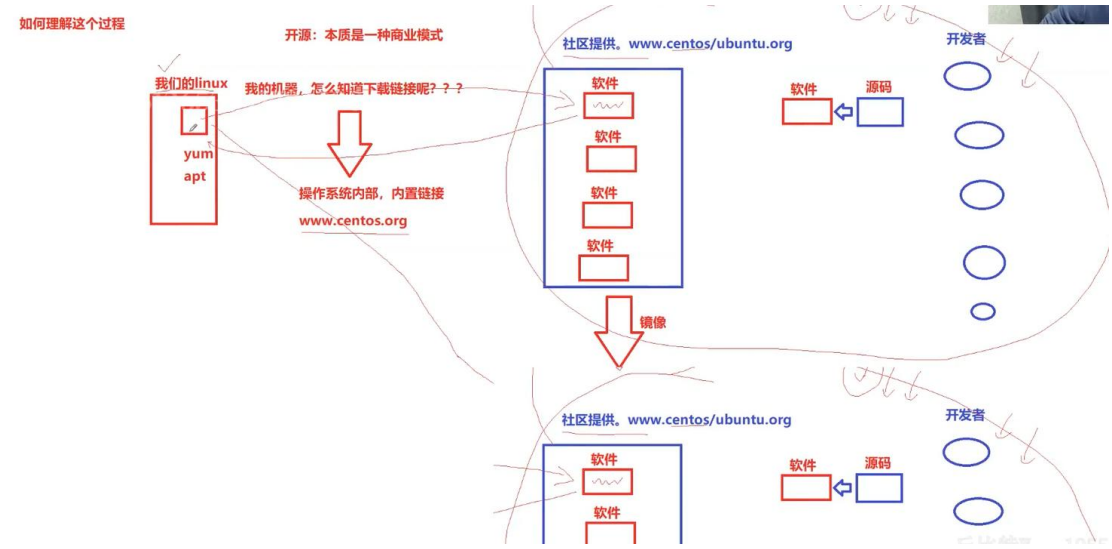


不推荐前两种：1 时操作麻烦繁琐 2 是软件版本和 Linux 内核版本可能不兼容或者不适配，有时候难以解决版本兼容问题

什么是包管理器呢？

类似手机上面的 应用商店





‘

把国外社区服务器里的资源拷贝一份到国内的镜像里，更改下载连接也就是更改镜像源

如何使用 yum 下载软件呢?

（注意：一般下载软件需要 root 权限，所以普通用户下载在自己的家目录里就需要用 sudo 来提权了）

sudo yum list ：查看可以下载的所有软件列表

sudo yum list | grep 关键字 可以查找包含关键字的软件列表

sudo yum remove -y 软件名 强制卸载这个软件

sudo yum install -y 软件名 强制下载这个软件

```
yum list | grep lrzsz
sudo yum remove lrzsz
sudo yum install lrzsz
```

/etc/yum.repos.d/* （yum源配置文件）

sudo yum -y install man-pages

base 稳定软件源，epel 扩展软件源

epel 下载：yum install -y epel-realse

如何更改配置源？自行查找资料

Vim

Vim 是一款编辑器

1 vim 初步

输入 vim 这个命令就直接进入了

退出; shift + ：然后输入 q 就退出了

也可以连按几次 shift+Z 也能退出

vim code.c 就打开 code 这个文件进行编辑，但如果没有配置这样写就跟记事本没什么区别

2 vim 的多模式

Vim 是一款多模式的编辑器

首次打开默认为命令模式：想正常写东西需要输入 i（也就是直接敲击键盘上的 i 即可）进入插入模式

进入插入模式可以任意写了。

在插入模式下退出需要回退到命令模式：直接按 esc 键就行

然后需要从命令模式进入到底行模式才能退出：shift 与 ： 同时按

然后输入 q 就能退出

在底行模式下输入 set nu 就能显示行号

从底行模式回到命令模式无脑 esc 就行了；但无法不通过命令模式从插入模式到底行模式

3 vim 的命令模式，底行模式等

Vim 的命令模式：gg 快速回归光标到第一行

Shift+g 快速把光标定位结尾

数字+shift+g 可以把光标定位到想到的一行

Shift+$ 可以把光标快速定位到这一行的行末

Shift+^ 可以把光标定位到这一行的开头

shift+x ：光标右侧不动，左侧删除

h 光标左移 j 下移 k 上移 l 右移

数字加上上面四个光标移动键对应往对应方向移动几个键

指令 w 以单词为单位向右移动

指令 b 以单词为单位向左移动

W 与 b 这两个指令同样能与数字结合使用

Linux 的 vim 指令模式下（一下操作均是指令模式下）

复制当前行是 yy 然后直接 p 就能粘贴

那如何撤销刚才的动作呢？按 u 就能直接撤销了；那如果我对这个 u 撤销后悔了呢？按 ctrl+R 就能撤销刚才的撤销

dd 剪切当前行；如果 dd 后不用 p，也就是可以删除当前行

yy p d 组合时同样可以在使用前加上数字

按 x 可以删除单个字符（删右侧）

shift+x ：光标右侧不动，左侧删除（删左侧）

同样的，操作后可以进行剪切，不是简单的删掉

注意：一旦退出文件编辑，就无法再进行撤销了，但如果只是保存没有进行 q 退出，可以撤销

也有单字符替换：按下 r +要替换的字符，可以把光标处的字符替换，同样可以与数字联动；

shift+r replace 模式：批量化替换，可以按行为单位去任意替换已经有的内容

无脑 esc 退出 replace 模式

shift+~ （大小写切换）会把小写的字符替换为大写的字符（反之也成立）；按着不动会把后面的都替换为大写/小写

ctrl+b 屏幕往“后”移动一页

ctrl+f 屏幕往“前”移动一页

ctrl+u 屏幕往“后”移动一页

ctrl+d 屏幕往“前”移动半页

在 vim 编辑器下，//就是注释符号，但如何进行多行等其他形式的注释，而不是一行一行的去标记//呢？

在命令模式下 ctrl+v 进入视图模式（esc 退出）

进入后，只能用 h j k l 来移动光标选择区域 （这里的移位操作仍然可以配合数字使用的）

选择好区域后，输入 shift+i 其实就进入插入模式了，然后输入// 再按 esc

就会发现选中的区域全被//注释掉了

又有问题来了：当我们这样注释后保存并且退出 vim 编辑后，再次进入 vim 编辑，想把刚才的注释去掉，却发现撤销不行后：继续 ctrl+v 想按照之前的操作删掉注释（选择区域，输入 d 即可删除），但发现只删掉了一列，而注释有两列；因为这样做只选中了一列信息，就不能只用 jk 上下移动了，应当适当调用 hl 来左右移动光标来选中多列

同时呢，在视图下，ctrl+I 的输入再 esc 的操作会导致选中的区域都有一样的输出，这也就是统一管理

在视图模式下，之前的光标操作基本上也都成立

讲了增删改了，有没有查呢？当然是有的：

shift+# 选中单词 会把光标处的单词识别出来，然后后面一样的单词都会高亮，想要跳转只要按 n 就会跳转到下一个单词处了

从命令模式进入插入模式的方法：

1 单点一下 i

2 单点一下　a　　同时也会使光标移动到上一行

3 单点一下　o　会使光标进入下一行，同时会新起一行

保存退出：底行模式下的 wq! 并 enter（w 保存，q 退出，感叹号代表强制性）

其实还有：shift（不动）+z+z

底行模式下的其他命令：

set nonu 去掉行号

set nu　加上行号

! 加上其他命令　,可以在不退出 vim 下进行编译运行等命令

%s/a /b/ 再回车 把 a 批量替换为 b

(未配置的)vim 文件名 然后 wq 保存退出 相当于生成一个新的空文件

vs 文件名　分屏打开两个文件　此时光标在哪一个窗口下代表在操作哪一个窗口

此时如果想切换光标　ctrl 然后 w 按两下即可切换(可以分很多屏)

使用 vim 的技巧

vim 文件名 +n　n 为数字,可以让打开文件时直接让光标到第 n 行

!单个字母　　可以使用最近一次的以单个字母开头的命令 最常用的就是这样快速打开之前用 vim 编辑的文件

4 vim 的配置

vim 的配置分为简单的配置和较为高级的配置

简单的配置: 主要是 vim 编辑器的一些知识:

.vimrc 是位于当前用户主目录下的隐藏文件，只对当前用户生效(也就是个人配置自个的 vim)。Vim 启动时会自动读取它。如果在用户目录下不存在,可以在用户目录的家目录下创建　　touch ~/.vimrc　　然后用 Vim 打开它进行编辑加入对应指令就完成了 vim 的初级配置

补充说明：除了用户级的 ~/.vimrc，系统还有一个全局配置文件（通常在 /etc/vim/vimrc 或 /etc/vimrc），对所有用户生效。个人的.vimrc 优先级更高，会覆盖全局设置。

高级配置:

高级配置就稍微复杂一点了,比如想要让 vim 能实现自动补齐这个功能时,就需要进行插件级别的配置

从简单的 set 命令到插件配置，是 Vim 从“好用的文本编辑器”迈向“智能开发环境”的关键一步。实现自动补全这类插件级功能，需要引入一个插件管理器，再由它来安装和驱动补全引擎。

第一步：安装插件管理器 vim-plug

手动管理插件（下载、解压、更新）非常繁琐，vim-plug 是目前最流行、简洁的插件管理器。

然后再进行类似安装等配置(可以去 gitee 或者 GitHub 搜索进行一键式自动化配置安装)

Linux 编译器 gcc/g++的使用

在运行代码时,程序会经过 预处理(进行宏替换)->编译(生成汇编)->汇编(生成机器可识别代码)->连接(生成可执行文件或库文件)

预处理:

gcc -E code.c -o code.i　会把预处理结果放到 code.i 这个临时文件里面(-E　:开始进程程序翻译,在预处理做完的时候就停下来)

在观察 code.i 的内容时,会发现:增加了很多内容,因为头文件要引用到程序里面,同时也有去注释,把注释去掉;宏也会发生替换(如 define M=100;　在 code.i 中就会发现找不到 M)

编译:

gcc -S code.i -o code.s (-S　:开始编译,编译做完了就停下来)

汇编:

gcc -c code.s -o code.o (code.o 为可重定位文件,在 Windows 的 vs 中常见为 code.obj　已经是二进制文件了　但还无法直接执行,我们的源文件中会包含很多库方法)　(-c　:开始翻译,汇编完成就停下来)

上面的三个选项简记为 esc　忘了看键盘,只要前两个大写就行 形成的文件后缀连起来就是 .iso

连接:

gcc code.o -o code

多文件连接:　先把这多个 code1.c code2.c 等文件用:　gcc -c code1.c　gcc -c code2.c　然后系统默认生成 code1.o code2.o　然后　gcc code1.o code2.o -o code　再./code　进行程序的输出

ldd code 可以查看可执行程序依赖哪些库

```
[whb@bite-alicloud lesson7]$ ldd code
        linux-vdso.so.1 =>  (0x00007ffe7e944000)
        libc.so.6 => /lib64/libc.so.6 (0x00007fb0f4794000)
        /lib64/ld-linux-x86-64.so.2 (0x00007fb0f4b61000)
```

其中 /lib64/libc.so.6 就是 c 语言标准库,用来连接 c 语言的一些库函数

补充:库

1.动态库: 在 Linux 中一般是.so 结尾　在 windows 中一般以.dill 结尾

2.静态库: 在 Linux 中一般是.a 结尾　在 Windows 中一般是.lib 结尾

补充:

gcc 可以进行命令行级别的宏定义:

gcc code.c -o code -DM　意思就是 define M